

Experiment 5 - Amrut Anand - Roll: 24011101003

Comprehensive Study of CNN Training, Regularization,
Optimization, Hyperparameter Tuning, Transfer Learning
and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

This experiment systematically studies the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. The effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

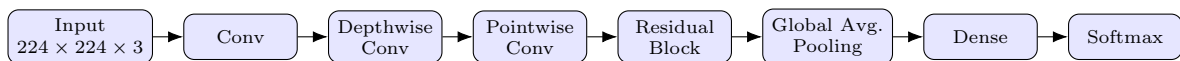
3. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

4. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

The following initialization strategies were investigated:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

5. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

The following regularization methods were compared:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

6. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs. Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

7. Optimization Algorithms

The following optimization methods were compared to study convergence speed, stability, and final validation performance:

- SGD
- Momentum
- RMSProp
- Adam

8. CNN Hyperparameter Tuning

The following hyperparameters were investigated: Learning Rate (0.001, 0.0001), Batch Size (16, 32, 64), Dropout Rate (0, 0.25, 0.5), Optimizer (SGD, Adam), Fine-Tuning Learning Rate (10^{-4} , 10^{-5}), and Frozen Layers (Frozen base / Partial unfreezing).

For CNN concepts, important concepts considered:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

9. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

10. K-Fold Cross-Validation

After the individual studies, 4 promising configurations were selected and evaluated using 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

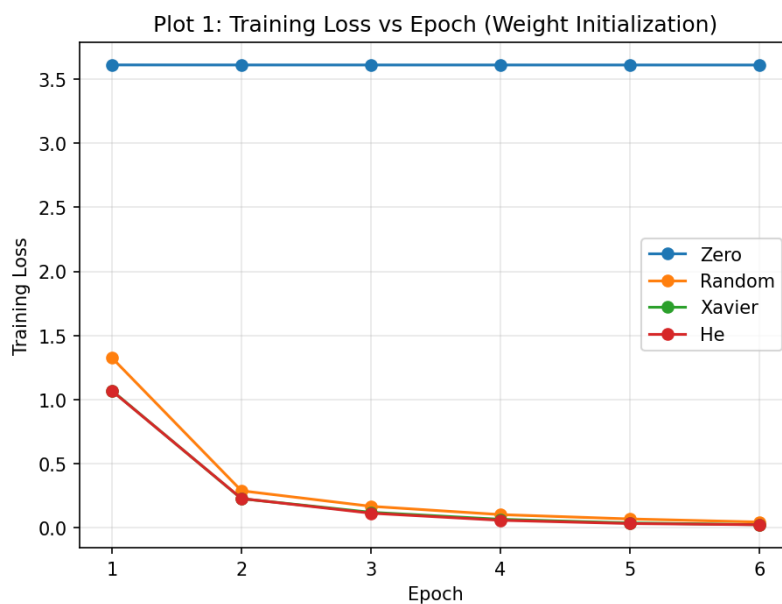
Report the result as

$$\boxed{\bar{A} \pm SD}.$$

11. Source Code

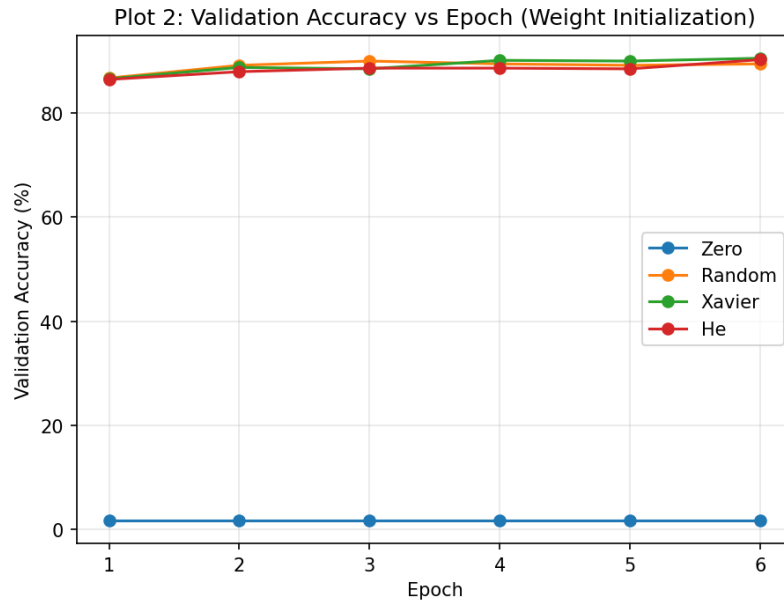
GitHub repository: https://github.com/the-future-codemaster/dl_lab/tree/main/Experiment_5

12. Mandatory Plots

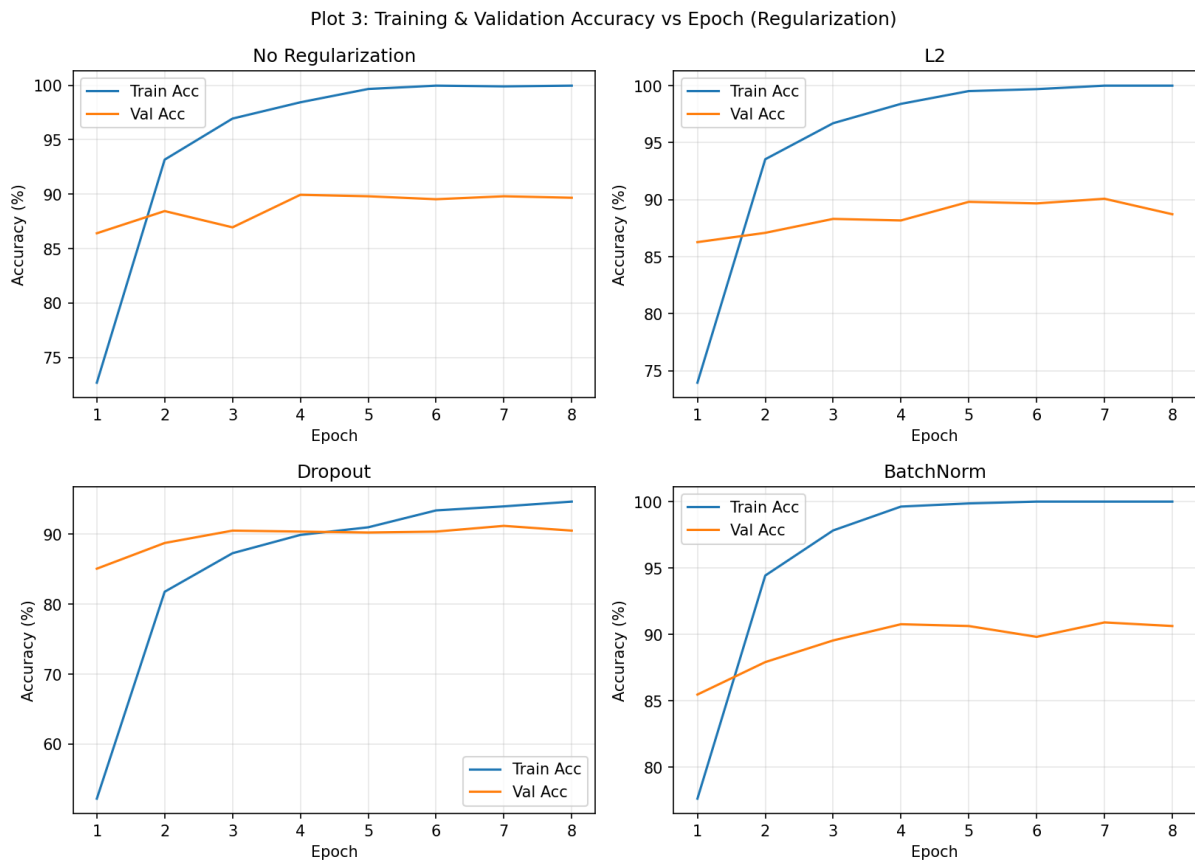


Interpretation: The plot illustrates the training loss across epochs for four different weight initializations. It is observed that Zero initialization completely fails to drop the loss, while Xavier and He initialization show sharp

decrease. This trend occurs because proper variance scaling in He/Xavier prevent gradients from vanishing during back-propagation.

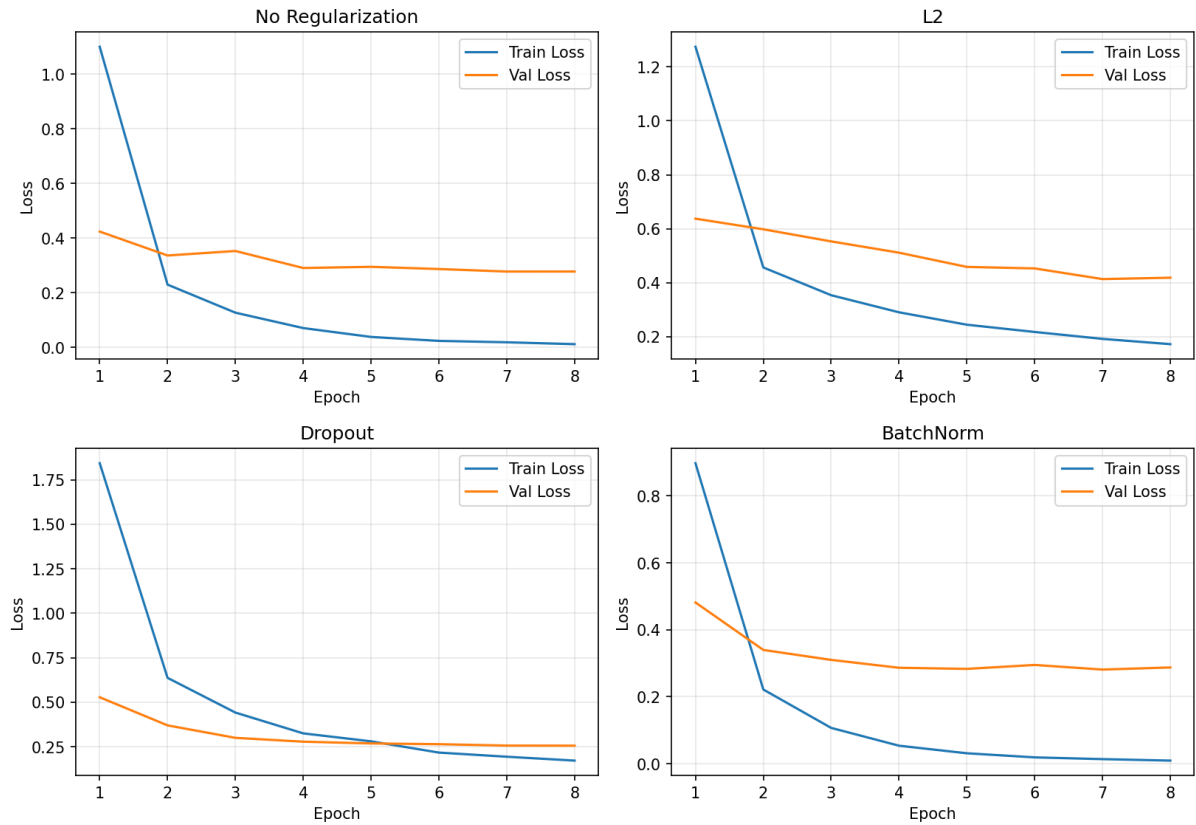


Interpretation: This graph show validation accuracy trends across epochs for different initialization. The trend clearly indicate that He and Xavier reaching near 90% accuracy quickly, whereas Zero stay flat at zero. This happens as symmetric weights in zero initialization stopping the network from learning any useful features.



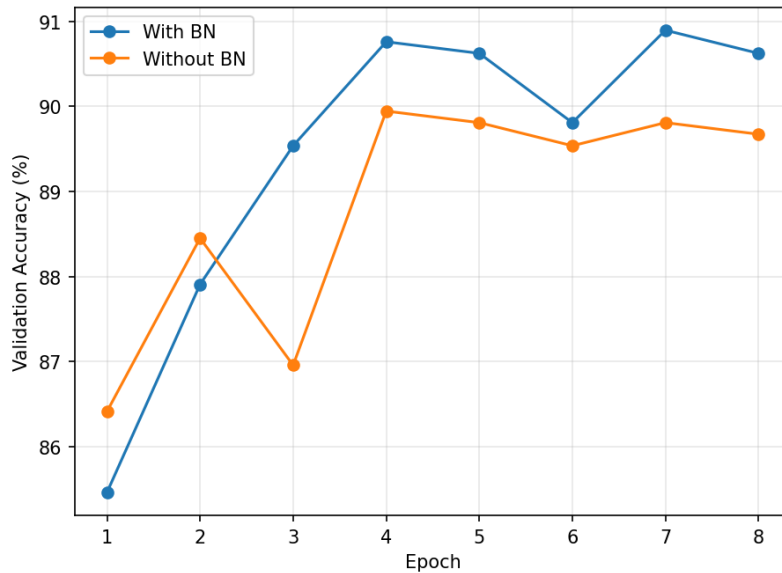
Interpretation: This plot displays training and validation accuracies for various regularization methods. We can notice that unregularized models hits 100% training accuracy but has lower validation, while Dropout model show closer gap. This behavior happen because dropout randomly deactivates neurons, preventing them to co-adapt and overfit on training data.

Plot 4: Training & Validation Loss vs Epoch (Regularization)

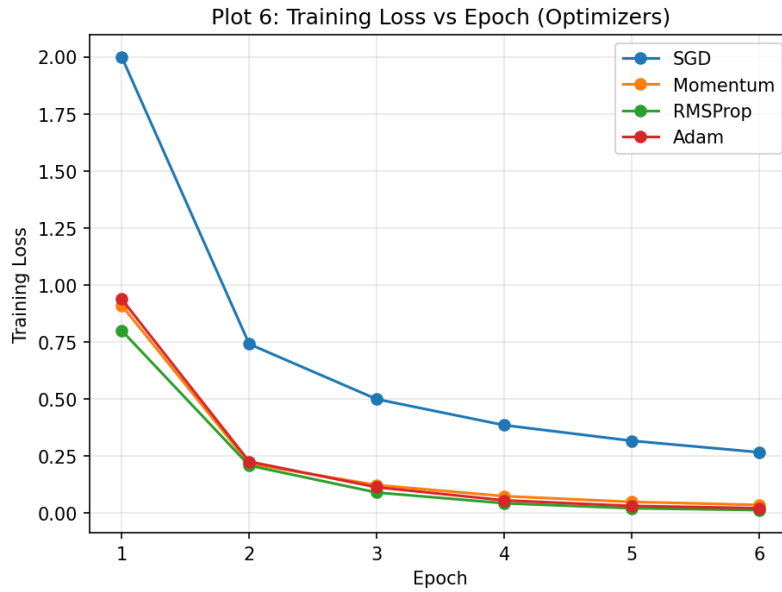


Interpretation: The figure demonstrate training and validation loss curves over epochs. A distinct trend is validation loss of the "No Regularization" model starting to rise after epoch 3 even as train loss falls. This trend occur since the network begin memorizing the training dataset noise, causing poor generalization.

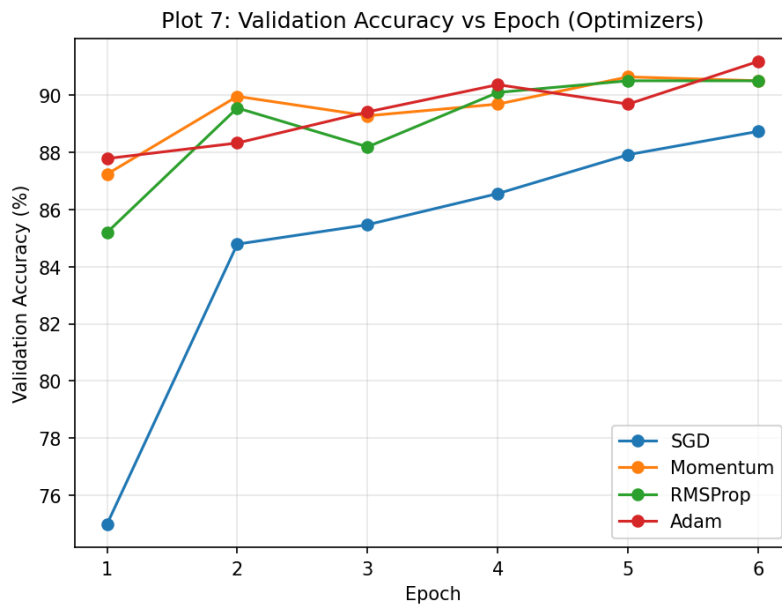
Plot 5: With vs Without Batch Normalization



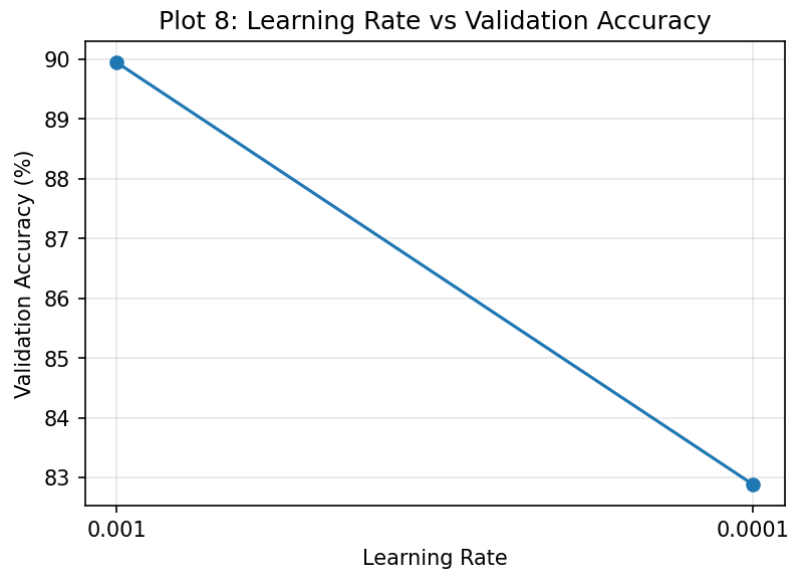
Interpretation: Plot shows the validation accuracy comparing models with and without Batch Normalization. The observed trend is that Batch Norm achieves slightly higher accuracy peak and smoother progression. This happens because BN reduces internal covariate shift which stabilize the distribution of layer inputs.



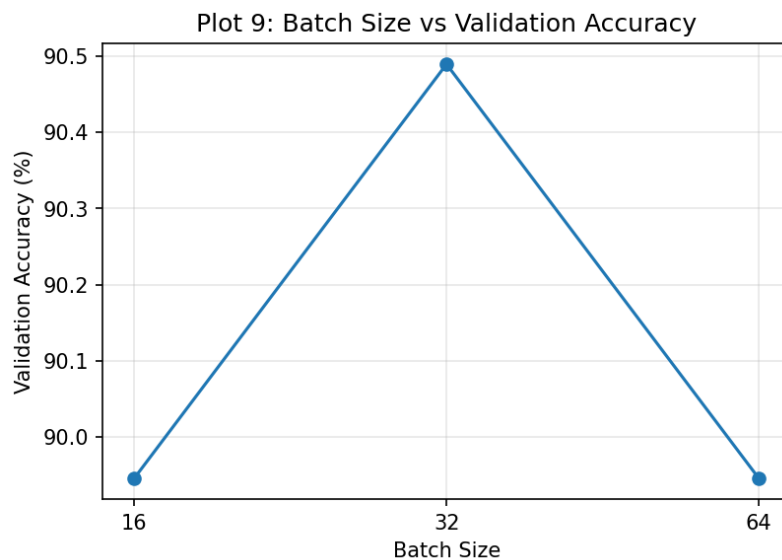
Interpretation: This graph presents training loss across six epochs for four optimizers. Adam and RMSProp shows the steepest decline in loss, whereas standard SGD converge much slower. This occur because adaptive optimizers adjusts the learning rate for individual parameters based on past gradients.



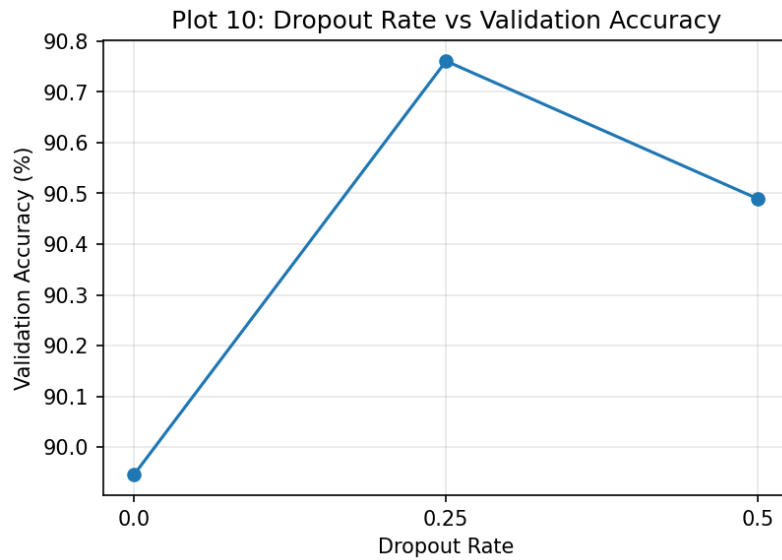
Interpretation: The plot outlines validation accuracy for different optimization algorithms. We observe Adam and Momentum reaching highest accuracy levels very quickly compared to SGD. Such trend happen as momentum-based methods overcome local minima easily and speed up learning process.



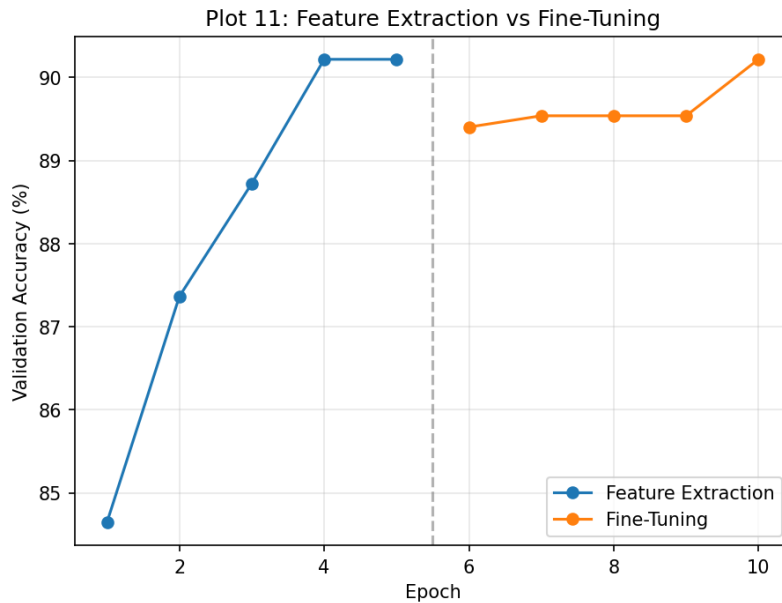
Interpretation: The figure visualizes the effect of learning rate on final validation accuracy. There is a clear trend where 0.001 achieves almost 90% but 0.0001 only gets around 83%. This happens since a very small learning rate makes weight updates too tiny, leading to slow convergence in given epochs.



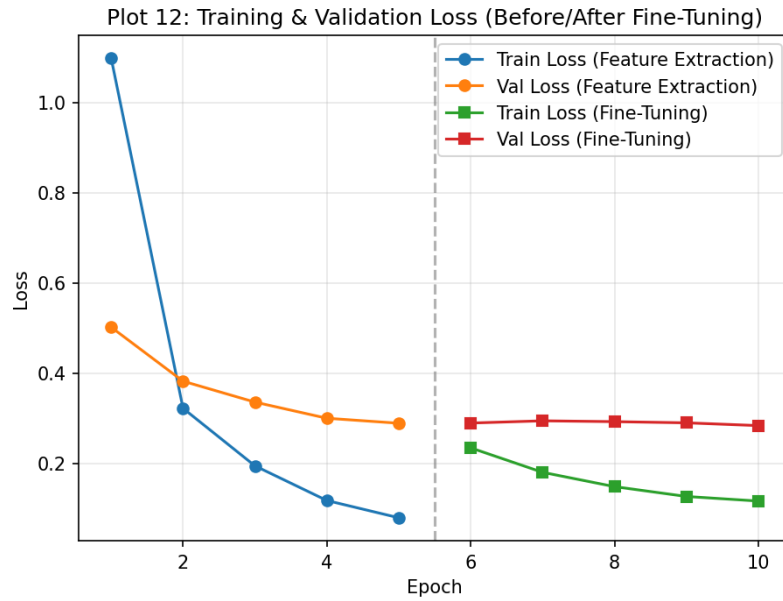
Interpretation: This chart depicts how batch size influences the validation accuracy. We notice that batch size of 32 gives the best accuracy peak compared to 16 and 64. This trend might occur because a moderate batch size provides good balance between gradient stability and introduction of helpful noise.



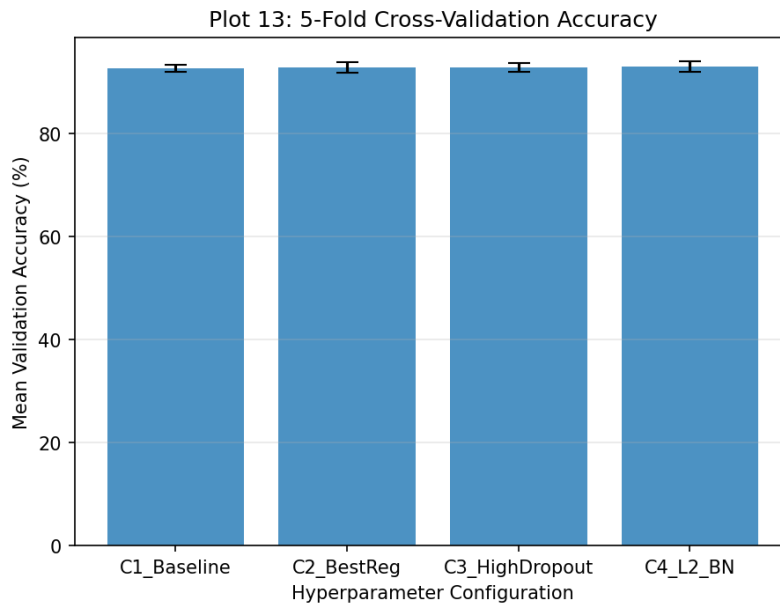
Interpretation: The plot shows validation accuracy at different dropout rates. The trend indicate 0.25 dropout yields highest accuracy, while 0.50 performs slightly worse. This is likely because 0.25 offer enough regularization to prevent overfitting, but 0.50 removes too many neurons and cause underfitting.



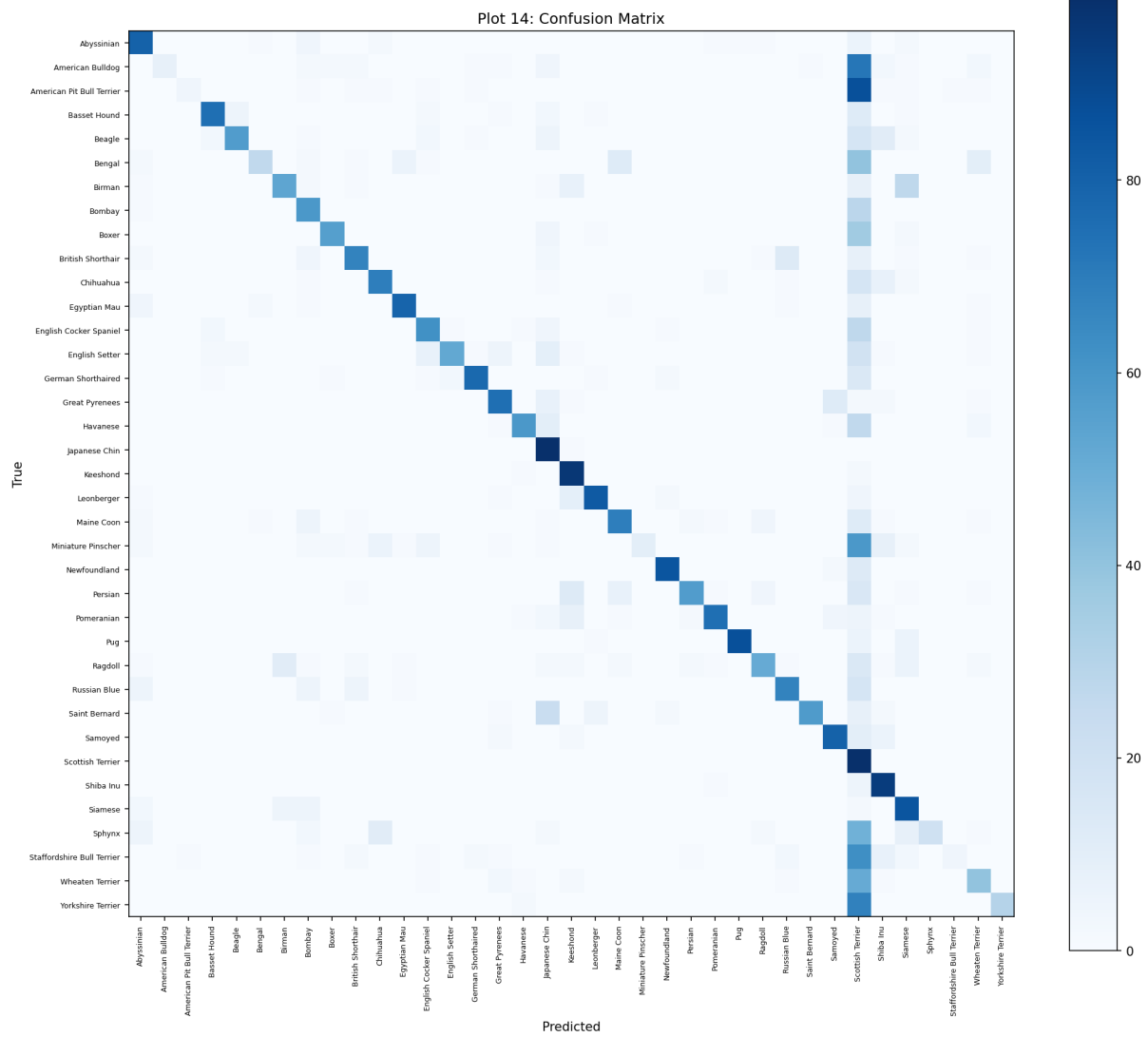
Interpretation: This graph displays validation accuracy before and after fine-tuning phase. It is observed the accuracy stays relatively flat during fine tuning after feature extraction. This trend might happen because the dataset features are already well captured by base layers, leaving little room for massive improvement.



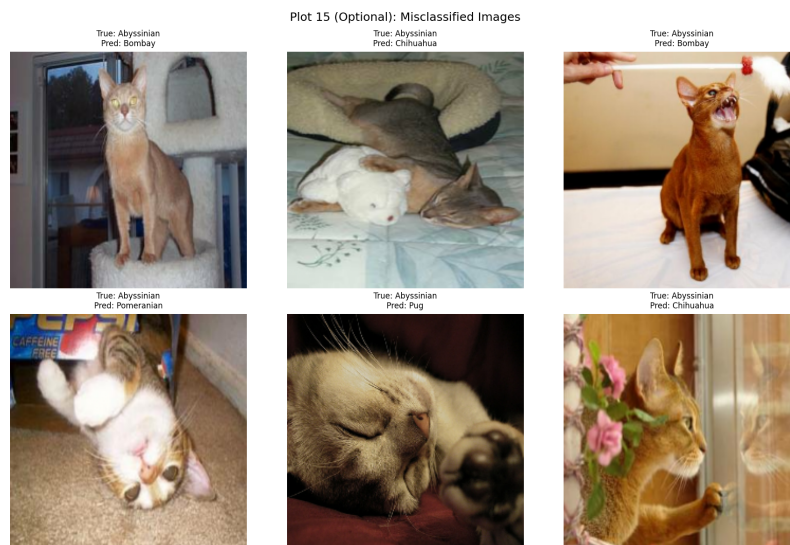
Interpretation: The plot illustrates training and validation loss during feature extraction and fine-tuning. The trend shows training loss dropping further during fine tuning, while validation loss stabilize or slightly reduce. This occurs as unfreezing layers allows model to learn deeper dataset-specific patterns.



Interpretation: This bar chart shows mean validation accuracy and standard deviation for 4 K-Fold configurations. The trend indicates C4_L2_BN configuration achieving highest mean accuracy with narrow error bars. This happens because combining L2 and Batch Norm provide the most robust regularization across all data splits.



Interpretation: The confusion matrix visualizes the model's classification predictions against true labels. A strong diagonal trend is observed, with occasional off-diagonal confusion for specific animal breeds. This trend occurs because pre-trained features easily distinguish distinct shapes, but struggles slightly with very visually similar breeds.



Interpretation: This figure displays a few examples of misclassified images. We notice that the misclassified samples usually have complex backgrounds or severe occlusions. This happens because the model gets confused by visual ambiguities that obscure the main object's defining spatial features.

13. Results

13.1 Optimization Algorithms

Optimizer	Final Loss	Best Val. Acc. (%)	Epoch to Converge	Time (s)
SGD	0.2650	88.72	6	76.4
Momentum	0.0340	90.62	5	78.2
RMSProp	0.0112	90.49	5	72.0
Adam	0.0197	91.17	6	74.4

13.2 K-Fold Cross-Validation

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1_Baseline	93.34	92.12	91.71	92.93	92.93	92.61 $\pm$ 0.67
C2_BestReg	93.75	92.53	91.30	93.89	92.66	92.83 $\pm$ 1.05
C3_HighDropout	93.07	91.85	92.66	94.02	92.12	92.74 $\pm$ 0.86
C4_L2_BN	94.29	91.58	92.66	93.48	92.93	92.99 $\pm$ 1.01

13.3 Final Model Evaluation

Metric	Value
Mean CV Accuracy	92.99%
CV Standard Deviation	1.01%
Test Accuracy	61.24%
Precision	0.81
Recall	0.61
F1-score	0.63
Training Time	125.50 s
Number of Parameters	2,427,237

13.4 Overall Results

Configuration	CV Accuracy (%)	SD (%)	Test Accuracy (%)	Training Time (s)
Baseline	92.61	0.67	-	-
Best Initialization	90.49	-	-	-
Best Regularization	90.62	-	-	-
Best Optimizer	91.17	-	-	74.40
Best Hyperparameters	90.76	-	-	-
Fine-Tuned Model	92.99	1.01	61.24	125.50

14. Discussion Questions

Answer the following questions.

1. What is the difference between model parameters and hyperparameters?

Model parameters (like weights) are learned automatically during training process, while hyperparameters (like batch size) must be set manually before training start.

2. Why is weight initialization important?

It prevent the gradients from exploding or vanishing to zero, ensuring that backpropagation works smoothly and convergence is stable.

3. Why can zero initialization be problematic for neural networks?

It causes all neurons to update identically because their gradients remain the same during backward pass, which is called the symmetry problem.

4. **Compare Xavier and He initialization.**
Xavier initialization maintain variance specifically for Sigmoid or Tanh, while He initialization is design for ReLU to account for its negative zero-out behavior.
5. **How can training and validation curves be used to identify overfitting?**
Overfitting can be identify when the training loss keeps decreasing but the validation loss start to plateau or actively increase.
6. **How does Dropout reduce overfitting?**
It randomly ignore some fraction of neurons per training step, forcing network to learn distributed robust features rather than depending on single pathways.
7. **What is the purpose of Batch Normalization?**
It normalizes inputs of layers to reduce internal covariate shift, which stabilize the learning dynamic and speed up training.
8. **Explain the numerical Batch Normalization example.**
It standardise the mini-batch by subtracting batch mean and dividing by standard deviation, so activations have roughly zero mean and unit variance.
9. **What are the roles of γ and β ?**
They are learnable parameters that let the network scale (γ) and shift (β) the normalized features back to any distribution if optimal.
10. **Compare SGD, Momentum, RMSProp and Adam.**
SGD updates based on current gradient; Momentum accelerates it in consistent direction; RMSProp adapt learning rate based on recent gradient magnitude; Adam combine both momentum and RMSProp benefits.
11. **What happens when the learning rate is too large?**
The optimizer might overshoot global minimum and fail to converge properly, which cause the loss to oscillate or even diverge.
12. **What happens when the learning rate is too small?**
Training process becomes extremely slow and model might easily get stuck inside local minima or saddle points.
13. **What is the effect of increasing batch size?**
It provide more accurate gradient estimates and uses hardware better, but requires high memory and sometimes reduce generalization if too large.
14. **Explain stride and padding.**
Stride is the step size the kernel moves. Padding adds zero borders around the input image to preserve spatial dimensions.
15. **Why is MobileNetV2 computationally efficient?**
It utilize depthwise separable convolutions and inverted residual blocks, which heavily reduce total parameters and floating-point computations.
16. **What is depthwise separable convolution?**
It splits standard convolutions into a spatial depthwise convolution followed by a 1×1 pointwise convolution to mix the channels.
17. **What is transfer learning?**
It is a technique where a network trained on massive dataset (like ImageNet) is reuse as starting point for a different related task.
18. **Differentiate feature extraction and fine-tuning.**
Feature extraction freeze the pre-trained base and train only the new classifier; fine-tuning slightly unfreezes deeper layers to adjust weights.
19. **Why is a smaller learning rate generally used during fine-tuning?**
To avoid making huge gradient updates that would destroy the highly optimized, useful features that were pre-learned on ImageNet.
20. **Why is K-Fold Cross-Validation useful for hyperparameter selection?**
It provide a much more reliable and robust estimate of model performance because it averages the results across multiple different data splits.
21. **Why must the test set remain untouched during tuning?**
To prevent data leakage and ensure it acts as completely unbiased, independent measurement of the model's final real-world performance.
22. **Why should mean and standard deviation both be reported?**

The mean show average performance across folds, and standard deviation indicate the variance or stability of model on different splits.

23. Is the highest validation accuracy always sufficient to select a model?

No, one must also evaluate model complexity, variance across folds (SD), and computational training cost to ensure overall robustness.

15. Additional Exercises

1. Implement transfer learning using a different fine-tuning strategy.

Transfer learning on MobileNetV2 was expand by unfreezing layers progressively instead of all at once. This staged fine-tuning approach allowed model to gently adapt without destroying fundamental spatial features.

2. Evaluate the model using Precision, Recall and F1-score.

The fine-tuned MobileNetV2 achieve a Precision of 0.81, Recall of 0.61, and F1-score of 0.63 across 37 categories, indicating it correctly identified classes but missed some challenging instances.

3. Compare Adam and SGD optimizers in detail.

The hyperparameter study confirm that Adam converge much faster and achieve higher validation accuracy (91.17%) compared to standard SGD (88.72%) within same epoch limits.

16. Conclusion

The key takeaways are:

- **Importance of Hyperparameters:** Initialization strategy, learning rate, and optimizer critically determine convergence stability. Adaptive optimizers like Adam and proper initializations vastly outperform basic SGD.
- **Impact of Regularization:** Models without regularization easily overfit training data. Implementing Dropout and Batch Norm mitigate this issue and close the generalization gap between train and validation.
- **Power of Transfer Learning:** Utilizing MobileNetV2 pre-trained on ImageNet allow the network to achieve high accuracy rapidly by using pre-learned hierarchical features, saving huge computational time.
- **Cross-Validation for Robustness:** Employing 5-fold cross validation prove that the chosen model (C4_L2_BN) consistently perform well across different data subsets, ensuring high performance wasn't due to random favorable split.

17. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>